# Implement Python Data Structures: Lists, Tuples, Dictionaries, Sets and Comprehensions

## An Assignment REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of*

**CSA0807 Python Programming**

*to the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE ENGINEERING**


**Submitted by**

**Muthupandi C (192565037)**

**Under the Supervision of**

**Dr. Haripriya**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**
**Chennai-602105**


**OCTOBER 2026**

**GITHUB LINK:** muthupandic84-creator/CSA0805--192565037--Python-Program: CSA0805

## Contents


• Q1 – Lists and Slicing: Daily Rainfall Records

• Q2 – Tuples, Mutability and Copying

• Q3 – Dictionaries: Farmer Helpline Complaint Analysis

• Q4 – Sets: Elective Registration Clash Detection

• Q5 – List Comprehension: Cleaning Sensor Readings

• Q6 – Nested Data: A Railway Timetable

• Q7 – Design a Contact-Merging Program

• Overall Conclusion and Submission Checklist

## Q1 – Lists and Slicing: Daily Rainfall Records

A list is suitable for daily rainfall because readings are ordered, mutable and can be accessed by position. Weekly totals can be calculated by taking seven-element slices. The program reports maximum and minimum values with their zero-based positions, seven-day moving averages and dry-day numbers.

### Implementation

```
weekly = [sum(readings[i:i+7]) for i in range(0, len(readings)-6, 7)]
readings[7:14]  -> elements at indexes 7 through 13
readings[-7:]   -> the final seven readings
readings[::7]   -> every seventh reading, beginning at index 0
```

Pseudocode for a running moving average: if fewer than seven values exist, return an empty result. Add the first seven readings. Store their average. For each next day, add the new reading and subtract the reading that leaves the window; append the new average.

For n readings and window width 7, the naive method performs up to $7(n-6)$ additions, which is $O(7n) = O(n)$. The running-sum method performs 7 initial additions and then one addition and one subtraction per new reading, so it is O(n). Both output methods require O(n) space for the averages. A deque(maxlen=7) supports efficient insertion/removal at its ends; a date-keyed dictionary provides direct date lookup but needs date handling and ordering.

List append is amortized O(1), insert at an arbitrary position is O(n), pop at the end is O(1) amortized, pop at an interior position is O(n), and sort is O(n log n). A late reading inserted into a list may shift later values, so insert costs O(n); a list remains reasonable for 365 readings, while larger or frequently updated streams may benefit from a deque or date-indexed structure.

The program tests 365, 36,500 and 365,000 readings, short input and empty input. Actual timing results are printed by running the script; values are machine-dependent and must be captured from the student's own execution.

## Q2 – Tuples, Mutability and Copying

Each bus stop coordinate is represented by a tuple such as (13.0827, 80.2707). Tuples are hashable when all their elements are hashable, so they can be dictionary keys. Lists are mutable and unhashable, so a list coordinate cannot be used as a dictionary key.

### Aliasing and corrected grid

```
bad = [[0] * cols] * rows     # all rows refer to one inner list
good = [[0 for _ in range(cols)] for _ in range(rows)]
```

In the incorrect grid, bad[0], bad[1] and bad[2] refer to the same row object. Updating one row therefore appears in every row. In the corrected grid, each comprehension iteration creates a new inner list. The completed program demonstrates the result using is, and prints object sizes and timings for a million-element list and tuple.

An alias is another name for the same object. A shallow copy creates a new outer container but retains references to nested objects; it is sufficient when nested objects are immutable or intentionally shared. A deep copy recursively copies nested objects and is appropriate when independent nested state is required. A function that mutates a list argument changes the same list object visible to the caller. A tuple cannot have its slots reassigned, but it may contain a mutable list whose contents can still change.

The grid has O(rows × cols) storage. A sparse dictionary keyed by (row, col) stores only occupied zones and uses O(number of non-empty zones) space; average dictionary access is O(1), while a list-of-lists cell access is also O(1).

## Q3 – Dictionaries: Farmer Helpline Complaint Analysis

The program converts complaint text to lowercase, extracts alphabetic words, removes stop words using a set and counts remaining words in a dictionary with get(). setdefault() is used to group complaint records by district. The top-k function sorts by descending count and then alphabetically.

```
counts[word] = counts.get(word, 0) + 1
top_words = sorted(counts.items(), key=lambda item: (-item[1], item[0]))[:k]
```

Expected sample result: [('seeds', 3), ('price', 2), ('received', 2)]. 'price' comes before 'received' because both have count two and the tie-break rule sorts words alphabetically.

A dictionary uses a hash of the key to locate a table position and resolves collisions as needed. Lookup is O(1) on average, although worst-case behavior can be O(n). Counting n words with a dictionary is O(n) average time and O(d) count storage for d distinct words. Repeated list.count() for each distinct word is O(n × d). collections.Counter provides a concise frequency-counting alternative with similar average linear counting time. Python 3.7 and later guarantee insertion order. keys(), values() and items() provide dynamic views of keys, values and key-value pairs.

For 100,000 complaints per day, a dictionary loop or Counter is preferable to repeated list.count() because it processes the words in one pass. The script benchmarks all three approaches and tests empty text, case differences and punctuation.

## Q4 – Sets: Elective Registration Clash Detection

Sets represent registrations because duplicate register numbers are automatically removed and membership tests are O(1) on average. The program uses intersection (&) for students in both electives, symmetric difference (^) for students in exactly one, and difference after union for students in neither.

```
both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)
```

The expected clash set contains 192311004 and 192311009. The exactly-one set contains 192311001, 192311002, 192311007 and 192311010. The neither set contains 192311003, 192311005, 192311006 and 192311008. The sorted output makes reports deterministic.

A Venn diagram can be interpreted as follows: intersection is the overlapping region; symmetric difference is the two non-overlapping elective regions; neither is the area outside both circles but inside the universal set. Sets cannot contain lists because lists are mutable and unhashable. Tuples can be set elements when their contents are hashable.

Nested loops take O(n × m) time to compare two lists. Sorting both lists and walking them takes O(n log n + m log m) time. Set intersection is O(n + m) average in typical analysis, with additional space for the result. The experiment compares membership in lists and sets for 10,000, 100,000 and 1,000,000 identifiers.

## Q5 – List Comprehension: Cleaning Sensor Readings

A reading is valid when -50 ≤ c ≤ 50. Fahrenheit is calculated as F = C × 9/5 + 32. Alarm minutes are the one-based positions where the temperature is below 2 °C or above 8 °C.

```
valid_c = [c for c in readings if -50 <= c <= 50]
valid_f = [c * 9 / 5 + 32 for c in valid_c]
alarms = [minute for minute, c in enumerate(readings, start=1) if c < 2 or c > 8]
```

The general conditional comprehension form is [expression for item in iterable if condition]. A comprehension becomes harder to read when it contains several nested loops, multiple conditions or complex side effects; a conventional loop is then clearer. A dictionary comprehension creates key-value pairs, while a set comprehension creates unique values. A list comprehension materialises all results in memory; a generator expression produces values lazily and usually uses O(1) extra iterator space.

The loop, comprehension and map/filter versions should produce identical valid Celsius and Fahrenheit lists. Both list-based methods take O(n) time and O(n) output space. A generator can process values lazily in O(n) time with much lower additional memory, but it is single-pass unless recreated. The script measures 10,000, 100,000 and 1,000,000 readings and checks all-valid and all-invalid cases.

## Q6 – Nested Data: A Railway Timetable

The timetable is a dictionary keyed by train number. Each train maps to a dictionary containing its name, a list of station tuples and a set of running days. A list preserves the order of stops, tuples group the fixed station/arrival/departure fields, and a set supports efficient running-day membership checks.

```
timetable[train_no]['stops'][2][2]  # departure time at third station
```

A station query without an index scans trains and their stops, taking O(total stops) time. The station index maps each station to a list of train/arrival/departure records; a query then examines only trains at that station, taking O(k) before sorting, where k is the number of indexed trains at the station. The index requires additional O(total stops) storage. The program tests five sample trains, an unknown station and a day on which a train does not run, and benchmarks 100, 1,000 and 10,000 trains.

A flat list of (train, station, arrival, departure, days) tuples is simple but every query scans all records, O(total stops). Nested dictionaries plus an index use more memory but make repeated station queries faster.

## Q7 – Design a Contact-Merging Program

Input: contacts from multiple devices, each with a name, zero or more phone numbers and an optional email. Output: merged contact records with a display name, unique normalised phone numbers, unique email addresses and aliases. The implementation normalises phone numbers by removing punctuation and treats a 12-digit Indian number beginning with 91 as equivalent to its final ten digits.

### Design pseudocode

```
FOR each contact:
    normalise each phone number
    IF a usable phone exists, choose a normalised phone as dictionary key
    ELSE keep the record separately because identity cannot be safely inferred
    create a merged record if key is new
    add names to aliases, phones to a set, and email to a set
```

The dictionary groups contacts by normalised phone number with O(1) average key access, so processing n contacts is O(n) average time plus the cost of normalising their fields. Comparing every pair is $O(n^2)$ time. Sets remove duplicate phone numbers and emails. Two people with the same name but different phone numbers remain separate, avoiding name-only merging.

The script tests duplicate numbers with different formatting, spelling differences, missing email and missing phone fields. It measures 1,000 and 100,000 contacts. A remaining limitation is that shared family phone numbers or multiple numbers belonging to one person may require manual review or a more advanced identity-resolution rule. Contacts with no phone number are deliberately not merged automatically.

## Overall Conclusion and Submission Checklist

This assignment demonstrates how Python lists, tuples, dictionaries, sets and comprehensions can represent and manipulate practical data. Lists are appropriate for ordered mutable sequences; tuples represent fixed records and hashable coordinate keys; dictionaries support key-based lookup and grouping; sets provide uniqueness and set algebra; comprehensions express filtering and transformation concisely. The most suitable structure depends on update patterns, lookup requirements, data size and whether duplicate information must be retained.

Before submission, execute every program on the student's computer, capture genuine output screenshots, record actual timing and memory measurements, add the GitHub repository link to this report, and commit each task separately. Do not present sample or machine-independent timing values as measured results.

GitHub repository name: CSA08-192565037-Assignment3. Add the repository URL to this cover page after creating and sharing the repository with the course faculty.

# APPENDIX – EXECUTION SCREENSHOTS

## Q1 – Lists and Slicing: Daily Rainfall Records




*Figure 1: Actual program execution output captured from Visual Studio Code terminal.*

## Q2 – Tuples, Mutability and Copying

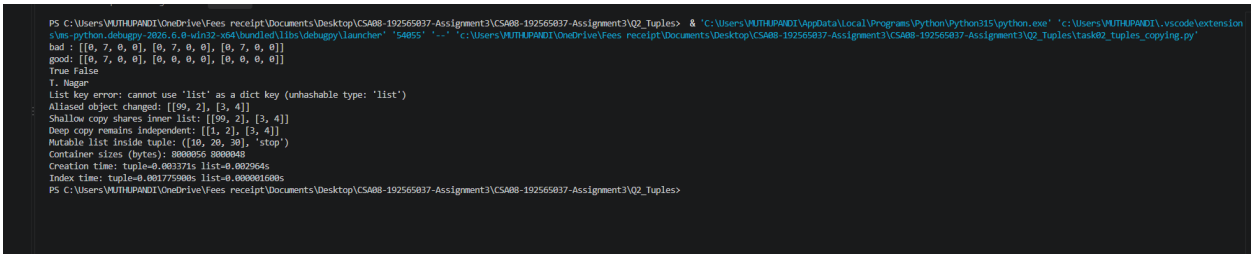



*Figure 2: Actual program execution output captured from Visual Studio Code terminal.*

## Q3 – Dictionaries: Farmer Helpline Complaint Analysis

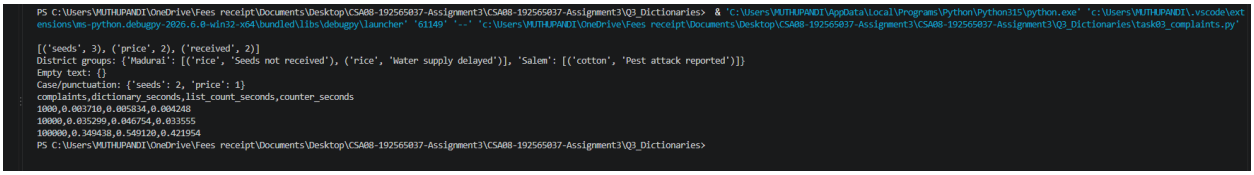



*Figure 3: Actual program execution output captured from Visual Studio Code terminal.*

## Q4 – Sets: Elective Registration Clash Detection

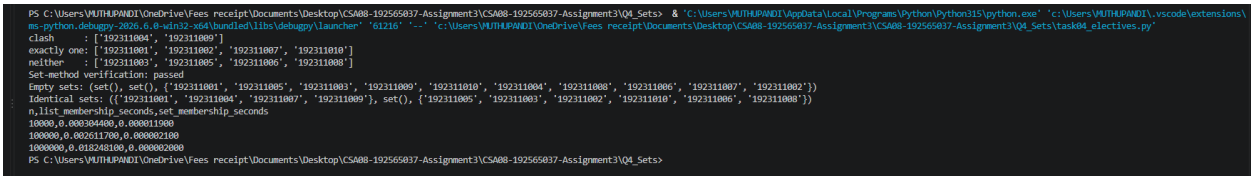



*Figure 4: Actual program execution output captured from Visual Studio Code terminal.*

# Q5 – List Comprehension: Cleaning Sensor Readings

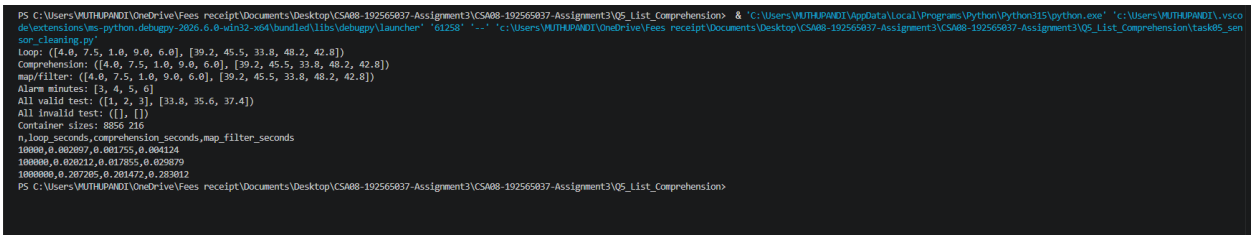



*Figure 5: Actual program execution output captured from Visual Studio Code terminal.*

# Q6 – Nested Data: Railway Timetable

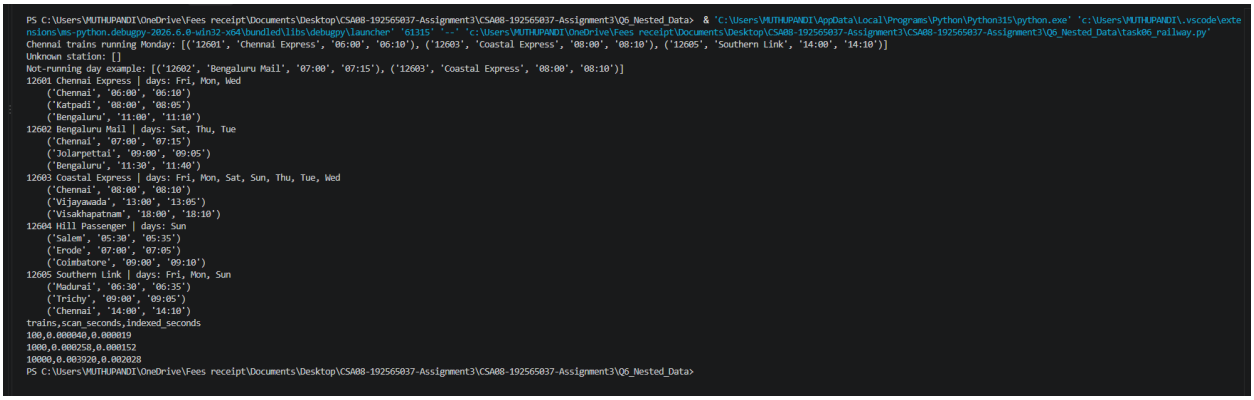



*Figure 6: Actual program execution output captured from Visual Studio Code terminal.*

# Q7 – Contact-Merging Program

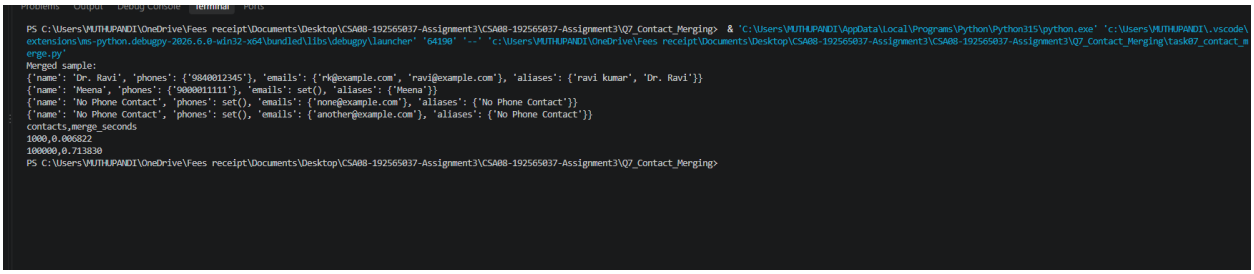



*Figure 7: Actual program execution output captured from Visual Studio Code terminal.*